COMPUTER LABORATORY

Assignment 6

Data Cleaning & Preparation

Telecommunications Customer Churn Analysis

VIVA PREPARATION GUIDE

Code walkthrough • Python & Pandas • preprocessing theory • cross-questions • common mistakes

What this experiment does

The notebook prepares a telecom customer dataset for machine-learning analysis. It removes duplicate customer records, standardizes text, converts the churn target into numbers, splits data into training and testing sets, fills missing values, limits extreme numeric values, creates a new feature, encodes categories, scales numeric features and exports the results.

Important: This guide follows the code in your uploaded notebook. Where the notebook has a potential issue or an improvement opportunity, it is clearly labelled so you can explain it confidently in a viva.

1. Problem statement & overall workflow

Customer churn means a customer stops using a company's service. A telecom company can study customer details such as tenure, monthly charges, total charges and service choices to understand or predict which customers may leave.

Why data preparation is needed

• Real-world data may contain duplicate rows, blank strings, missing values and inconsistent text.

• Machine-learning algorithms generally need clean numeric input rather than arbitrary text labels.

• Features measured on very different scales can affect some algorithms, so scaling may be useful.

• A clean train/test split helps estimate performance on unseen customers.

Notebook workflow

Step	What the notebook does
1	Import Pandas, NumPy and scikit-learn tools
2	Read CSV and preview records
3	Count/remove duplicate customer IDs; clean text; encode Churn
4	Convert selected columns to numeric
5	Split into training (80%) and testing (20%)
6	Impute missing values using training-set statistics
7	Detect IQR bounds and clip extreme charge values
8	Create NewCustomer feature
9	Combine cleaned train/test rows for export and inspection
10	One-hot encode categorical variables
11	Scale numeric columns using StandardScaler
12	Export cleaned data and model-ready train/test files

Cell 0 – importing libraries

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

pandas (pd): library for tabular data; DataFrame is like a spreadsheet. **NumPy (np):** numerical computing library; imported here but not used in the shown notebook cells. **StandardScaler:** standardizes numeric features. **train_test_split:** randomly divides data into training and testing subsets.

2. Loading, checking and cleaning the data

Cell 1 – read the CSV

```
customers = pd.read_csv('Telecom_Customer_Churn.csv')
customers.head()
```

read_csv() reads a comma-separated file and returns a DataFrame. **head()** displays the first five rows by default; it is a quick visual check, not a cleaning operation.

Cell 2 – count duplicate customer IDs

```
print(customers['customerID'].duplicated().sum())
```

duplicated() marks repeated values after the first occurrence as True. **sum()** counts True values because True behaves like 1. This checks repeated IDs, not necessarily every identical row.

Cell 3 – remove duplicates and normalize text

```
customers = customers.drop_duplicates(subset='customerID').copy()
text_columns = customers.select_dtypes(include=['object','string']).columns
for col in text_columns:
customers[col] = customers[col].astype('string').str.strip().replace('', pd.NA)
customers['gender'] = customers['gender'].str.lower()
```

drop_duplicates(subset=...) keeps the first row for each customer ID. **copy()** makes a separate DataFrame. **select_dtypes()** selects text-like columns. **astype('string')** converts values to Pandas' nullable string type. **str.strip()** removes leading/trailing spaces. **replace('', pd.NA)** turns empty strings into proper missing values. **str.lower()** makes gender text lowercase, helping standardize categories.

Convert target label

```
customers['Churn'] = customers['Churn'].str.lower()
customers['Churn'] = customers['Churn'].map({'yes':1, 'no':0})
customers['Churn'] = customers['Churn'].astype(int)
customers = customers.dropna(subset='Churn')
```

Churn is the target/label: Yes becomes 1 (customer left), No becomes 0 (customer stayed). **map()** replaces values using a dictionary. The notebook then casts the result to integer and attempts to remove missing target rows.

3. Numeric conversion, splitting & missing values

Convert numeric-looking columns

```
numeric = ['tenure', 'MonthlyCharges', 'TotalCharges']
customers[numeric] = customers[numeric].apply(
pd.to_numeric, errors='coerce').astype(float)
display(customers[numeric].isna().sum())
```

apply() applies a function to each selected column. **pd.to_numeric()** converts values into numbers. With **errors='coerce'**, values that cannot be converted become NaN instead of stopping the program. **astype(float)** stores the columns as floating-point numbers. **isna().sum()** counts missing values in each selected column.

Cell 4 – split into train and test

```
train, test = train_test_split(
customers, test_size=0.2, random_state=42,
stratify=customers['Churn'])
train = train.copy()
test = test.copy()
```

test_size=0.2 reserves 20% for testing and leaves 80% for training. **random_state=42** makes the random split reproducible. **stratify=Churn** preserves approximately the same churn Yes/No proportion in both sets. **copy()** creates independent copies.

Cell 5 – fill missing values

```
medians = train[numeric].median()
categorical = train.select_dtypes(include=['object','string']).columns
train[numeric] = train[numeric].fillna(medians)
train[categorical] = train[categorical].fillna('Unknown')
test[numeric] = test[numeric].fillna(medians)
test[categorical] = test[categorical].fillna('Unknown')
```

median() calculates the middle value for each numeric column. **fillna()** replaces missing values. Numeric columns use training medians; text columns use the category 'Unknown'. The same training medians are applied to test data to avoid learning preprocessing values from the test set.

Viva caution: target conversion order

Potential bug in the notebook: if Churn contains a blank or unexpected value, map() produces NaN, but astype(int) is executed before dropna(). That conversion can raise an error. Safer order: map labels, drop rows where Churn is missing, then convert the remaining labels to integer. If the source data guarantees only yes/no values, this may not appear during execution.

4. Outliers, feature engineering & checking

Cell 6 – IQR method and clipping

```
for column in ['MonthlyCharges', 'TotalCharges']:
q1 = train[column].quantile(0.25)
q3 = train[column].quantile(0.75)
iqr = q3 - q1
lower = max(0, q1 - 1.5 * iqr)
upper = q3 + 1.5 * iqr
train[column] = train[column].clip(lower, upper)
test[column] = test[column].clip(lower, upper)
```

quantile(0.25) gives Q1 (25th percentile); **quantile(0.75)** gives Q3 (75th percentile). **IQR = Q3** – **Q1**. The usual outlier fences are Q1 – 1.5×IQR and Q3 + 1.5×IQR. The code prevents the lower bound from going below zero using max(0, ...). **clip()** caps values below the lower bound and above the upper bound; it does not delete rows.

The bounds are calculated from train only, then reused for test. This is important because test data should not influence preprocessing decisions.

Cell 7 – create a feature

```
train['NewCustomer'] = (train['tenure'] < 12).astype(int)
test['NewCustomer'] = (test['tenure'] < 12).astype(int)
```

This creates a binary feature: tenure below 12 months gives 1; otherwise 0. The comparison returns True/False, and astype(int) converts True to 1 and False to 0. This is feature engineering: deriving a potentially useful variable from existing columns.

Cell 8 – recombine and inspect

```
cleaned = pd.concat([train, test]).sort_index()
display(cleaned.head())
display(cleaned.isna().sum())
```

concat() joins the rows of train and test. **sort_index()** restores original row-index order. **isna().sum()** checks remaining missing values column-wise. This combined DataFrame is for the cleaned-data export; the model-ready training and testing sets remain separate.

Important distinction

Combining the sets for a final cleaned CSV does not mean the model should train on the combined data. Training and evaluation must still use their separate subsets.

5. Encoding, scaling and exporting

Cell 9 – one-hot encoding

```
X_train = pd.get_dummies(
train.drop(columns=['customerID', 'Churn']), dtype=int)
X_test = pd.get_dummies(
test.drop(columns=['customerID', 'Churn']), dtype=int)
```

drop(columns=...) removes customerID (an identifier, not a useful general predictor) and Churn (the answer/target, which must not be included in input features). **get_dummies()** converts categories into separate 0/1 indicator columns. For example, Contract values Month-to-month, One year and Two year can become three columns. **dtype=int** stores indicators as integers.

Potential improvement: train and test may contain different categories, so get_dummies() can produce different columns. Before modelling, align them, for example by reindexing X_test to X_train.columns with fill_value=0. A fitted encoder such as OneHotEncoder(handle_unknown='ignore') is another robust approach.

Cell 10 – inspect dimensions

```
print('Training shape:', X_train.shape)
print('Testing shape:', X_test.shape)
```

shape returns (number of rows, number of columns). It helps verify that the split and encoding produced expected dimensions.

Cell 11 – standard scaling

```
scaled_columns = numeric
scaler = StandardScaler()
X_train[scaled_columns] = scaler.fit_transform(X_train[scaled_columns])
X_test[scaled_columns] = scaler.transform(X_test[scaled_columns])
```

StandardScaler transforms each feature approximately to mean 0 and standard deviation 1: z = (x – mean) / standard deviation. **fit_transform()** learns mean and standard deviation from training data and transforms it. **transform()** applies those same learned values to test data. Never fit the scaler separately on test data.

Cell 12 – save CSV files

```
cleaned.to_csv('06_cleaned_telecom.csv', index=False)
X_train.assign(Churn=train['Churn']).to_csv('06_train.csv', index=False)
X_test.assign(Churn=test['Churn']).to_csv('06_test.csv', index=False)
```

to_csv() writes a DataFrame to CSV. **index=False** avoids saving the Pandas row index as an extra column. **assign()** adds the Churn target back to the feature DataFrame for supervised learning. The notebook exports a cleaned combined dataset and separate model-ready train/test files.

6. Theory concepts you should know

Concept	Simple explanation
Data cleaning	Finding and correcting/removing inaccurate, incomplete, duplicated or inconsistent data.
Data preprocessing	Preparing raw data so it is suitable for analysis or machine learning. It can include cleaning, encoding, scaling and splitting.
Missing value	A value that is absent or unknown. Pandas commonly represents it as NaN or pd.NA.
Mean vs median imputation	Mean is the arithmetic average and is sensitive to extreme values. Median is the middle value and is usually more robust to outliers.
Categorical variable	A value representing a group or label, such as gender, payment method or contract type.
Numerical variable	A measurable quantity, such as tenure, monthly charges or total charges.
Outlier	An observation unusually far from most other observations. It may be an error or a genuine rare case.
IQR	Interquartile range, Q3 minus Q1; it measures the spread of the middle 50% of data.
Clipping / capping	Replacing values outside chosen bounds with the nearest bound; rows are retained.
Feature engineering	Creating useful input variables from existing data, such as NewCustomer from tenure.
Label / target	The value a supervised model is intended to predict. Here it is Churn.
Data leakage	When information from test/future data improperly influences training or preprocessing, making evaluation too optimistic.
One-hot encoding	Representing each category using a separate binary indicator column.
Standardization	Rescaling numeric values using training mean and standard deviation.
Normalization	A broad term for rescaling; min-max normalization often maps values to a fixed range such as 0 to 1.
Train/test split	Separating data for model learning and final evaluation on unseen examples.
Class imbalance	When one target class occurs much more often than another. Stratification preserves class proportions in the split.
Identifier	A field used to identify a record, such as customerID; it is usually removed from predictive features.

7. Viva questions – fundamentals & code

Q. What is customer churn?
Answer: When a customer stops using or paying for a company's service.

Q. What is the objective of this experiment?
Answer: To clean and prepare telecom customer data for further analysis or churn prediction.

Q. What is a DataFrame?
Answer: A two-dimensional labelled table in Pandas with rows and columns.

Q. Why use read_csv()?
Answer: To load a CSV file into a Pandas DataFrame.

Q. What does head() return?
Answer: The first five rows by default; head(10) returns the first ten.

Q. Why check duplicate customerID?
Answer: A customer should not be counted multiple times unintentionally.

Q. What does duplicated().sum() mean?
Answer: Count of values marked as duplicates after their first occurrence.

Q. Why use drop_duplicates(subset='customerID')?
Answer: To retain one record per customer ID.

Q. Why call copy()?
Answer: To create a separate copy and reduce accidental view/assignment issues.

Q. What does select_dtypes() do?
Answer: Selects columns based on data type, such as object or string.

Q. Why strip whitespace?
Answer: 'Male' and ' Male ' should not become separate categories.

Q. Why replace empty strings with pd.NA?
Answer: An empty string is not automatically treated as a missing value by all Pandas operations.

Q. Why convert Churn Yes/No to 1/0?
Answer: Most machine-learning workflows require numeric target labels.

Q. What does map() do?
Answer: Maps each value using a dictionary or mapping function.

Q. What is errors='coerce'?
Answer: Invalid numeric text is converted to NaN instead of raising a conversion error.

Q. Why inspect isna().sum()?
Answer: To see how many missing values each column contains.

Q. Why split before imputation?
Answer: To calculate imputation values only from training data and avoid leakage.

Q. Why use stratify?
Answer: To preserve approximately the same churn class ratio in train and test.

Q. Why random_state=42?
Answer: To make the random split reproducible; 42 is just a chosen seed.

Q. Why use median instead of mean?
Answer: Median is less affected by extreme values.

Q. Why fill categories with Unknown?
Answer: It preserves rows and explicitly marks missing category information.

Q. What does quantile(0.25) mean?
Answer: The first quartile: about 25% of values are at or below it.

Q. Does clipping remove outlier rows?
Answer: No. It caps their values but keeps the rows.

Q. What does astype(int) do?
Answer: Converts values to integer type; True becomes 1 and False becomes 0.

Q. Why drop customerID from X?
Answer: It is an identifier and generally does not describe customer behaviour in a reusable way.

Q. Why drop Churn from X?
Answer: To prevent the answer from being included as an input feature; that would be target leakage.

Q. What does get_dummies() do?
Answer: Converts categorical columns into 0/1 indicator columns.

Q. What does shape show?
Answer: The number of rows and columns.

Q. What does StandardScaler do?
Answer: Centers numeric features and scales them to unit standard deviation.

Q. Difference between fit_transform and transform?
Answer: fit_transform learns parameters and applies them; transform applies already learned parameters.

8. Cross-questions, pitfalls & last-minute revision

Q. Why calculate medians from train and not the full dataset?
Answer: Using the full dataset lets test information influence training preparation. Train-only statistics avoid this leakage.

Q. Why calculate IQR bounds on train only?
Answer: For the same reason: test data must remain unseen during fitting/preprocessing.

Q. Why use the same scaler on test data?
Answer: The model expects test features to be represented using the training scale.

Q. What if test has a category not seen in train?
Answer: The notebook's separate get_dummies calls can produce mismatched columns. Align columns or use a fitted encoder that handles unknown categories.

Q. Can customerID be useful?
Answer: It can be useful for record tracking or joining data, but usually should not be a predictive feature.

Q. What is the difference between dropna and fillna?
Answer: dropna removes rows/columns with missing values; fillna replaces missing entries.

Q. What if an entire numeric column is missing in train?
Answer: Its median is NaN, so median imputation cannot provide a meaningful value; use a deliberate fallback or remove/review the column.

Q. Does StandardScaler make data normally distributed?
Answer: No. It changes mean and standard deviation, not the distribution shape.

Q. Is standardization always necessary?
Answer: No. It is especially useful for scale-sensitive methods such as logistic regression, SVM and KNN; tree models often do not require it.

Q. What is the difference between scaling and encoding?
Answer: Encoding converts categories to numeric representation; scaling changes the scale of numeric values.

Q. Why preserve train and test separately after creating cleaned?
Answer: The combined file is convenient for a cleaned-data copy, but model training and evaluation must remain separate.

Q. What does assign(Churn=...) do here?
Answer: Adds the original target column back to the features so each exported dataset contains both predictors and label.

Q. What is a possible issue with astype(int) on Churn?
Answer: If map() created NaN for an unexpected label, integer conversion can fail before dropna runs. Drop missing labels first.

Q. What is the target variable in this notebook?
Answer: Churn.

Q. Does this notebook train a churn prediction model?
Answer: No model training or evaluation is shown in the supplied notebook; it prepares and exports the data.

Fast revision – remember these

• Churn: Yes → 1, No → 0; target column is not part of X.

• Split: 80% train / 20% test; random_state=42; stratify by Churn.

• Missing numeric values: training median. Missing categorical values: 'Unknown'.

• Outliers: IQR fences; clip MonthlyCharges and TotalCharges.

• Feature engineering: tenure < 12 creates NewCustomer (1/0).

• Encoding: get_dummies(). Scaling: StandardScaler fitted on train only.

• Exports: 06_cleaned_telecom.csv, 06_train.csv, 06_test.csv.

Three improvements to mention if asked

1. Move dropna(subset='Churn') before astype(int) to handle unexpected target labels safely. 2. Align encoded test columns to training columns (or use OneHotEncoder in a fitted preprocessing pipeline). 3. Consider using a Pipeline/ColumnTransformer so imputation, encoding and scaling are applied consistently during future model training.

One-sentence conclusion: “In this experiment, I cleaned telecom customer data, handled missing and extreme values, engineered a feature, encoded categorical columns, standardized numeric features and saved separate training and testing datasets for future churn modelling.”

Prepared from the uploaded Assignment_6.ipynb and the lab schedule entry for Data Cleaning and Preparation (Experiment 6).